Epistemic Credit - Speed - Quality Triangle

Relationship Interaction Analysis

Edge Relationships: Core Interactions

The following table analyzes the three edges of the triangle, showing how each pair of vertices interact:

Relationship	When You Increase...	Impact On...	Mitigation Strategies
Epistemic Credit ↔ Speed **Type: TRADE-OFF** **Inverse relationship**	↑ Speed (faster code generation) • Generate code rapidly with LLMs • Skip deep analysis • Minimal documentation • Accept AI suggestions quickly	↓ Epistemic Credit • Developers understand less • Gap between complexity & comprehension • "Working code without ownership" • Technical decisions become opaque	• Inverse Documentation (explain before commit) • Mandatory code review with explanation • "No explanation, no merge" rule • Break work into smaller chunks • Regular epistemic debt audits • Pair programming with LLMs • Documentation-first workflow
	↑ Epistemic Credit (deeper understanding) • Write tests first (TDD) • Document intent before coding • Review and refactor AI code • Team knowledge sharing	↓ Speed • Takes longer to ship features • More time in planning and review • Iterative refinement cycles • Knowledge transfer overhead • But: Faster long-term maintenance	• Strategic trade-off based on domain • Accept slower speed in core domains • Use DDD to focus understanding • Ubiquitous language reduces overhead • Automated testing compensates over time
Speed ↔ Quality **Type: TENSION** **Can conflict but not always inverse**	↑ Speed (rush to ship) • Skip comprehensive testing • Minimal edge case handling • Accept first working solution • Reduce code review rigor	↓ Quality (potentially) • More bugs and edge cases missed • Security vulnerabilities (45% of LLM code) • Brittle tests • Technical debt accumulation • Higher defect rates	• TDD (tests as specifications) • Property-based testing • Integration testing • Static analysis + fuzz testing • Multi-model review • Security-specific review • Automated quality gates • Deterministic verification points
	↑ Quality (rigorous testing) • Comprehensive test coverage • Property-based testing • Security audits • Multiple review passes	→ Speed (can improve with confidence) • Initially slower • But: Higher confidence enables faster iteration • Fewer production issues • Less debugging time • Paradox: Quality enables speed via reduced rework	• Invest in test infrastructure upfront • Automated testing pipelines • Clear quality gates prevent rework • "Shift left" - catch issues early • Quality becomes speed multiplier over time
Quality ↔ Epistemic Credit **Type: SYNERGY**	↑ Quality (good tests & verification) • Comprehensive test suites • Well-documented test cases • Clear property specifications	↑ Epistemic Credit • Tests serve as living documentation • Understanding emerges from TDD • Failed tests reveal comprehension gaps	• Maximize synergy through: • TDD as primary workflow • Tests as specifications • Behavior-driven development (BDD) • Document intent in test names

Mutually reinforcing	• Integration tests with real scenarios	• Property-based tests clarify invariants • Team learns system through tests	• Use tests to onboard developers • "Tests as prompt" for LLM generation • Explanation-forcing review of failures
	↑ Epistemic Credit (deep understanding) • Team understands domain deeply • Clear mental models • Documented architectural decisions • Ubiquitous language established	↑ Quality • Better design decisions • More thorough edge case coverage • Anticipate failure modes • Architectural coherence • Understanding enables better validation	• Domain-Driven Design (DDD) • Ubiquitous language in code and tests • Bounded contexts for clarity • Event storming to surface understanding • Code reviews as knowledge sharing • Architecture decision records (ADRs) • Understanding drives quality improvement

Context-Dependent Strategies

How to balance the triangle based on domain type and development phase:

Context	Epistemic Credit Priority	Speed Priority	Quality Priority
Core Domain (Strategic differentiator)	CRITICAL - Maximum investment • Deep TDD with integration tests • Strong DDD practices • Intensive code review • Low tolerance for epistemic debt • Team must fully understand • Extensive documentation	MODERATE - Deliberate pace • Accept slower feature delivery • Focus on sustainable velocity • Long-term speed through understanding • Trade short-term for maintainability	CRITICAL - Highest standards • Comprehensive testing • Property-based testing • Security audits • Multi-model review • Zero tolerance for known bugs
Supporting Domain (Enables core)	MODERATE - Balanced approach • Selective TDD on key paths • DDD concepts applied • Standard code review • Moderate epistemic debt OK • Focus understanding on interfaces	MODERATE - Reasonable velocity • Balance speed with quality • LLM assistance with guardrails • Pragmatic iteration • Avoid over-engineering	HIGH - Strong testing • Integration tests mandatory • Automated quality gates • Basic security review • Reliability is important
Generic Domain (Commodity/utility)	LOW - Accept debt strategically • Minimal TDD (smoke tests only) • Standard patterns, no DDD • Light code review • High epistemic debt tolerable • Use standard solutions • Heavy LLM assistance OK	HIGH - Maximize velocity • Rapid iteration priority • Heavy LLM generation • Minimal ceremony • Get to "good enough" quickly • Time to market critical	MODERATE - Basic gates • Basic functional testing • Automated smoke tests • Standard security checks • "Good enough" quality • Reliability important but not critical
Exploration Phase (Learning/prototyping)	LOW - Intentional debt • Skip deep understanding • "Vibe coding" acceptable • Minimal documentation • Learn through experimentation • Epistemic debt is temporary • Plan to rewrite anyway	CRITICAL - Maximum speed • Rapid experimentation priority • Fail fast, learn fast • Heavy LLM assistance • Minimal process • Optimize for learning speed	LOW - Just enough to validate • Basic functionality only • Manual testing acceptable • Focus on "does it work" • Quality improves in next iteration
Production/KTLO (Live system)	CRITICAL - Mandatory comprehension • Full TDD required • Strong DDD if complex • Rigorous code review • Zero tolerance for new debt • On-call team must understand • Comprehensive documentation	LOW - Controlled change • Stability over velocity • Careful, measured changes • Thorough testing before deploy • Risk mitigation paramount • Speed is secondary to reliability	CRITICAL - Maximum rigor • Comprehensive test coverage • Integration + E2E tests • Security scanning • Performance testing • Canary deployments • Rollback plans • Quality is non-negotiable

Strategic Insights

1. The Fundamental Trade-off

The Epistemic Credit ↔ Speed relationship is the core tension in LLM-augmented development. More speed almost always means less understanding initially, but strategic mitigation can minimize this trade-off.

2. Quality as the Mediator

Quality practices (especially testing) can transform the triangle dynamics. Good tests both verify correctness AND build understanding, creating a virtuous cycle that can ultimately improve speed through reduced rework.

3. Context Determines Balance

There is no single "correct" position on the triangle. The optimal balance shifts based on:

- Domain importance (core vs. supporting vs. generic)
- Development phase (exploration vs. production)
- Team capabilities and maturity
- Risk tolerance and business constraints

4. Deliberate Debt is Different

Just as with technical debt, epistemic debt taken consciously and strategically (e.g., in generic domains or exploration) is fundamentally different from accidental epistemic debt accumulation. The key is awareness and intentionality.

5. Long-term vs. Short-term Optimization

The triangle looks different across time horizons:

- Short-term: Speed can come at the expense of both quality and understanding
- Medium-term: Quality investment reduces debugging time, enabling speed
- Long-term: Understanding enables both quality and sustainable speed

Use the triangle visualizations to identify your current position and determine the optimal balance for your context